ISW (Resumen primer parcial)

Que es el software?:

El software es toda información o conocimiento en un producto, el software nunca es igual es personalizado. Set de programas y documentación (conocimiento empaquetado) que acompaña.

Caracteres escenciales del software:

- Complejidad: Escalar el software aumenta los elementos interactuando de forma no lineal
- Conformidad: Debe adaptarse a interfaces humanas arbitrarias y sistemas sin principios, dictados por el entorno
- Mutabilidad: El software es infinitamente maleable y contantemente sometida a la presion del cambio de continuo
- Invisibilidad: El software carece de representación espacial o geometerica. No puede dibujarse en un plano. Priva a la mente de la abstracción visual

Proceso de software:

Conjunto de actividades, metodos, practicas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados.

- Proceso unificado desarrollo (PUD): Especificación de requerimientos (ERS), Analisis, diseño, construcción , prueba y despliegue



Que es la ingenieria de software?:

La ingenieria de software es la aplicacion de un enfoque sistematico, disciplinado y cuantificable al desarrollo del software. **Es una disciplina que se encarga de todos los aspectos de la producción del software, tambien se encarga del mantenimiento despues de la operación**.

¿Por qué deberíamos gestionar la configuración del software?:

La configuración del software es un conjunto de items de configuración con su correspondiente version en un momento dado.

Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida

Una disciplina que aplica dirección y monitoreo administrativo y técnico para: identificar y documentar las características funcionales y técnicas de los ítems de configuración; controlar los cambios de esas características; registrar y reportar los cambios y su estado de implementación; y verificar la correspondencia con los requerimientos.

- Identificar características técnicas y funcionales de ítems de configuración;
- Documentar características técnicas y funcionales de ítems de configuración;
- Controlar los cambios de esas características identificadas y documentadas;

- Registrar y reportar estos cambios;
- Verificar correspondencia con los requerimientos (trazabilidad).

Items de configuración:

Todo archivo virtual, que sea del software. Ej: casos de uso, codigo, diseño de bases de datos, etc. Cada item de configuración debe tener un nombre unico y una versión, tuebe qye existir la posibilidad de volver a una versión.

Versión:

Una versión se definde, desde el punto de vista de la evolución; como la forma particular de un artefacto en un instante o contexto dado.

Variante:

Versión de un item de configuración que evoluciona por separado, las variantes de representan configuraciones alternativas. Un producto de software pjuede adopatar distintas formas dependiendo el lugar donde se lo instale.

Repositorio:

Es donde se almacenan los items de configuración, puede ser una o varias bases de datos.

Lineas base:

Una configuración que ha sido revisado formalmente y sobre la que se ha llegado a un acuerdo, sirve como base para desarrollos posteriores y puede cambiarse solo atravez de un procedimiento formal de control de cambios. Permite ir atras en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto

Filosofía agil, manifiesto agil:

valores agiles

- Individuos en interacciones, por sobre procesos y herramientas: es preferible un equipo de personas motivadas con un buen funcionamiento y conocimiento, donde la comunicación sea fluida, pero con herramientas

pobres para su trabajo, el desarrollo de software es una actividad humano-intensiva, y los procesos ágiles intentan capitalizar las fortalezas de cada individuo.

- Software funcionando por sobre documentación extensiva: Tras cada iteración puede ir mostrandose a los usuarios finales y obtener una retroalimentación de ellos, para que el equipo de desarrollo se asegure de estar trabajando siempre en aquellas funcionalidades de mayor valor para el cliente.
- Colaboración con el cliente por sobre negociación contractual: Implica comprometer al cliente como parte importante del producto que se está construyendo. Sobre todo el momento de probarlo y dar el feedback
- Responder a cambios por sobre seguir un plan: es imposible que los usuarios sepan con detalle y certeza todas las funcionalidades que necesitan, y a su vez, con el paso del tiempo surgirán nuevas necesidades. Desarrollar el incremento genera cierto conocimiento en el equipo, que afecta a la planificación en los distintos niveles. De forma similar, mostrar o desplegar esos incrementos a los usuarios finales, también generará nuevo conocimiento para esos procesos de planificación.

Los 12 principios del mainifiesto agil:

1- Nuestra mayor prioridad es satisfacer al cliente

2- Aceptar que los requisitos cambien

3- Entregar software funcional frecuentemente

4- Tecnicos y no tecnicos trabajando juntos durante todo el proyecto

5- Desarrollamos proyectos en torno a individuos motivados

6- El metodo mas eficiente de comunicar informacion es cara a cara

7- La mejor metrica de progreso es el software funcionando

8- Los procesos agiles promueven el desarrollo sostenible

9- La atención continua a la excelencia tecnica y al buen diseño mejora la agilidad

10- la simplicidad es escencial

11- Las mejores arquitecturas, diseños y requerimientos surgen de equipos autoorganizados

12- A intervalos regulares, el equipo evalúa su desempeño y ajusta la manera de trabajar

Filosofia agil:

Agil no es metodologia o proceso, el objetivo primordial es entregar el software de forma frecuente en un etorno cambiante, el software no se desarrolla como una sola unidad sino como un a serie de incrementos y cada uno de ellos incluye una nueva funcionalidad del sistema. El cliente se encuentra inmerso en el proceso. La filosofia agil esta orientada a las personas mas que a los procesos.

Es un balance entre ningún proceso y demasiado proceso. La diferencia inmediata es la exifencia de una menor cantidad de documentación, sin embargo no es eso lo importante.

Principio Lean:

Busca maximizar el valor generado al cliente con el minimo consumo de recursos, todo esfuerzo que no cree valor es desperdicio.

Atender al cliente final con alta calidad, bajo costo y tiempo de entrega, produciendo exactamente lo que el cliente final quiere, cuando lo quiere, donde quiere, costo minimo y precio justo.

User stories:

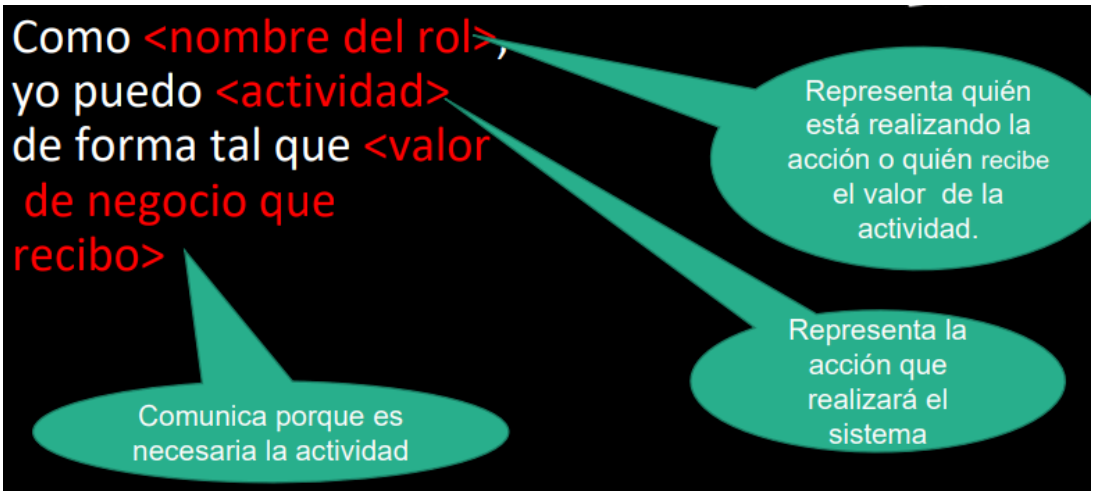
Las user stories son requerimientos de usuario alineados a un requerimiento de negocio. Al usuario le interesa el requerimiento de negocio. Pero el requerimiento de usuario es el medio por el cual vamos a lograr ese valor de negocio, los User Stories no sirven para especificar requerimientos de sofrware, sirven para identificar requerimientos de usuario.

- No son especificaciones detalladas de requerimientos (como los casos de uso)

- Son expresiones de intención, “es necesario que haga algo como esto...”
- No están detallados al principio del proyecto, elaborados evitando especificaciones anticipadas, demoras en el desarrollo, inventario de requerimientos y una definición limitada de la solución.
- Necesita poco o nulo mantenimiento y puede descartarse después de la implementación.
- Junto con el código, sirven de entrada a la documentación que se desarrolla incrementalmente después.

Partes de las User stories:

- Tarjeta: descripciónd e la historia, utilizada para planificar y como recordatorio. Tiene una sintaxis partucular que es la siguiente:



- Conversación: Es la parte más importante de la US, ya que explicita la comunicación entre el Product Owner y el Equipo de desarrollo.
- Confirmación: Conforma a la definición de las pruebas de aceptación. El PO decide si las pruebas de aceptación son válidas.

Criterios de aceptación de Users Histories:

Es una nota no obligatoria que define un acuerdo respecto a cómo se debe comportar el software para que el PO acepte la implementación, es información concreta que tiene que servirnos a nosotros para saber si lo que implementamos es correcto o no.

- Definen una intención, no una solucion
- Son independientes de la implementación
- Relativamente del alto nivel, no es necesario que se escriba cada detalle

Spikes

Tipo especial de historia, utilizado para quitar riesgo e incertidumbre de una User Story u otra faceta del proyecto.

Se clasifican en : técnicas y funcionales.
• Pueden utilizarse para:
• Inversión básica para familiarizar al equipo con una nueva tecnología o dominio.
• Analizar un comportamiento de una historia compleja y poder así dividirla en piezas manejables.
• Ganar confianza frente a riesgos tecnológicos, investigando o prototipando para ganar confianza.
• Frente a riesgos funcionales, donde no está claro como el sistema debe resolverla interacción con el usuario para alcanzar el beneficio esperado.

Técnicas
• Utilizadas para investigar enfoques técnicos en el dominio de la solución.
• Evaluar performance potencial
• Decisión hacer o comprar
• Evaluar la implementación de cierta tecnología.
• Cualquier situación en la que el equipo necesite una comprensión más fiable antes de comprometerse a una nueva funcionalidad en un tiempo fijo.

Funcionales
• Utilizadas cuando hay cierta incertidumbre respecto de cómo el usuario interactuará con el sistema.
• Usualmente son mejor evaluadas con prototipos para obtener realimentación de los usuarios o involucrados.

- Algunas User Stories requieren de ambos tipos de spikes. Por ejemplo: Como un cliente, quiero ver mi uso diario de energía en un histograma, para poder comprender rápidamente mi consumo de energía pasado, presente y proyectado.

• En este caso un equipo puede crear dos spikes:
• Spike Técnico:
• Investigar cuanto tiempo requiere actualizar un display de un cliente al uso actual, determinando requerimientos de comunicación, ancho de banda y si los datos se actualizan en formato push o pull.
• Spike Funcional:
• Crear un prototipo de histograma en el portal web y obtener la retroalimentación de algunos usuarios respecto del tamaño, el estilo de la presentación y los atributos gráficos.

ESTIMACIONES AGILES



La Dinámica Destructiva de la Subestimación.



1. Cronograma irreal dictado por el negocio.

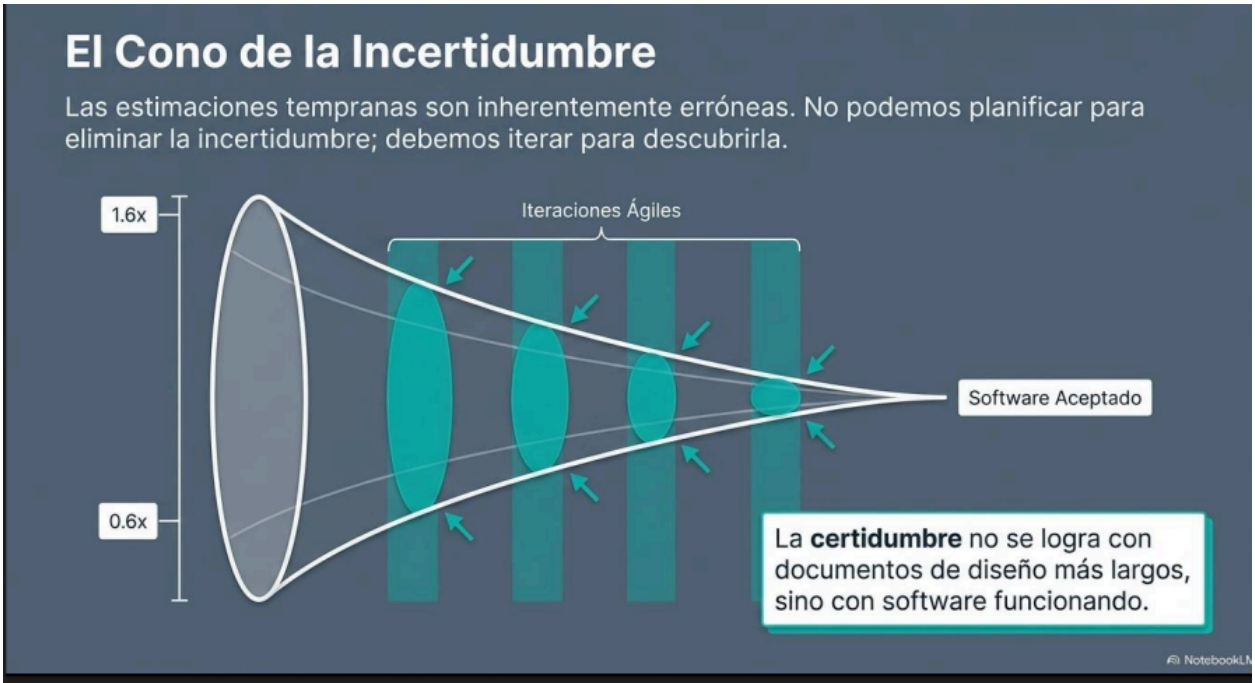
2. Se recortan actividades tempranas (Análisis, Diseño).

3. Se programan soluciones rápidas y sucias ('Quick & Dirty').

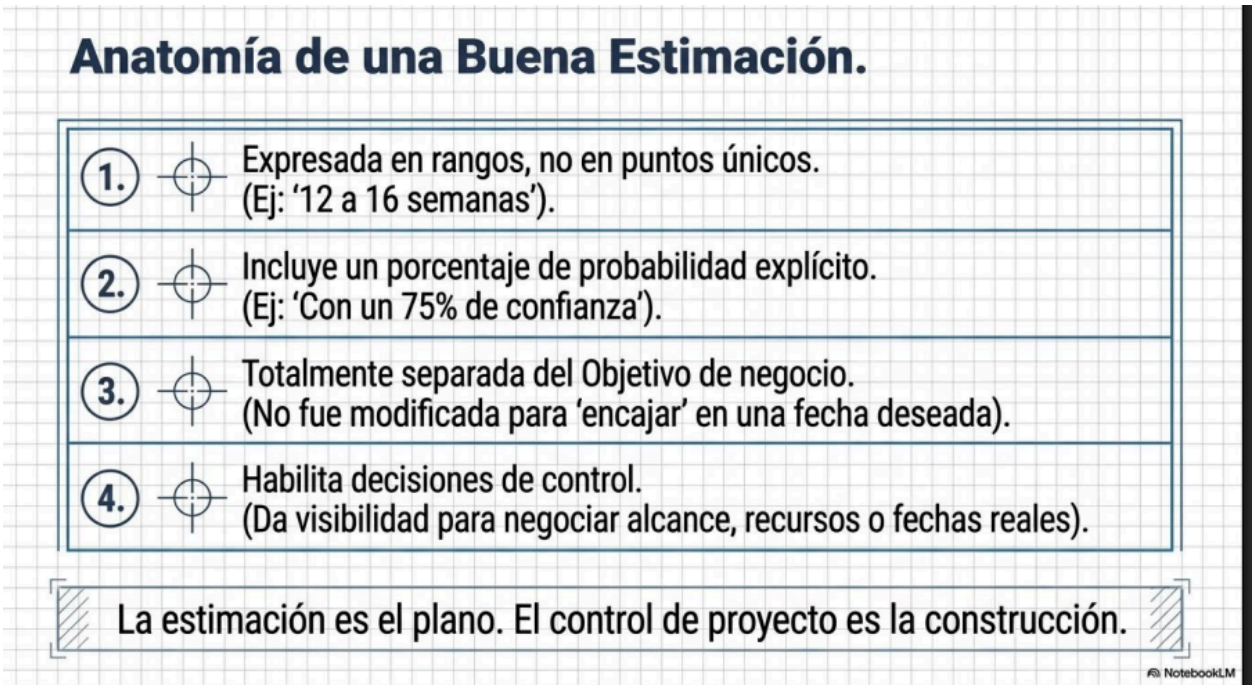
4. Alta tasa de defectos en pruebas o producción.

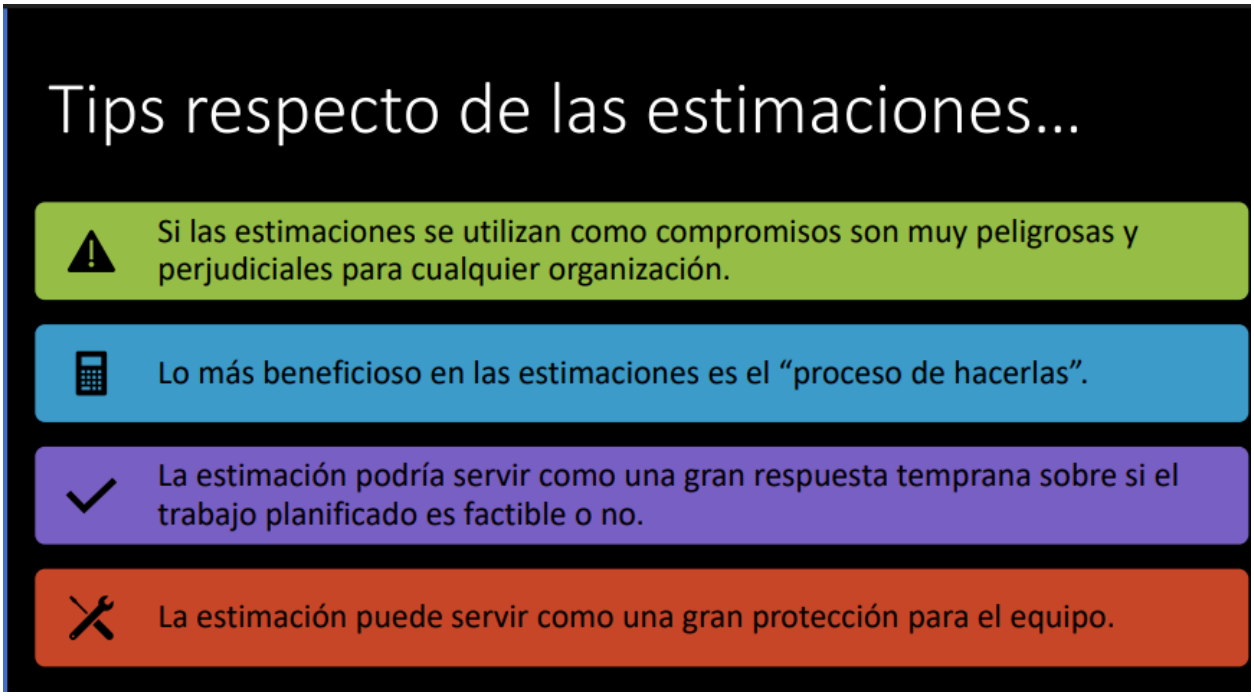
5. Reuniones de crisis, refactorización tardía y desvío de personal.

El apuro es el mayor enemigo de la ingeniería de software.


El Cono de la Incertidumbre
Las estimaciones tempranas son inherentemente erróneas. No podemos planificar para eliminar la incertidumbre; debemos iterar para descubrirla.
1.6x
0.6x
Iteraciones Ágiles
Software Aceptado
La certidumbre no se logra con documentos de diseño más largos, sino con software funcionando.


El cono no se estrecha por sí solo.
[Decisiones de Diseño]
[Definición de UI]
[Alcance Cerrado]





Las features/stories son estimadas usando una medida de tamaño relativo conocida como story points (SP), Las medidas relativas no son absolutas. Story Points no es una medida basada en tiempo.

Estimación Relativa

- Las personas no saben estimar en términos absolutos

• Somos buenos comparando cosas
• Comparar es generalmente más rápido.
• Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual
• Se emplea mejor el tiempo de análisis de las storys

Usamos la escala de Fibonacci (1, 2, 3, 5, 8, 13) para reflejar que a mayor tamaño, mayor es la incertidumbre

El tamaño es una medida de la cantidad de trabajo necesaria para producir una feature/story.
• El tamaño indica:
▪ Cuán compleja es una feature/story
▪ Cuánto trabajo es requerido para hacer o completar una feature/story
▪ Y cuán grande es una feature/story

Y qué es un Story Point?
• Es una unidad de medida específica (del equipo) de, complejidad, riesgo y esfuerzo, es lo que “el kilo” a la unidad de nuestro sistema de medición de peso
• Story point da idea del “peso” de cada story y decide cuan grande (compleja) es
• La complejidad de una feature/story tiende a incrementarse exponencialmente.

Velocidad (Velocity)
• Velocidad/ Velocity es una medida (métrica) del progreso de un equipo. Se calcula sumando el número de story points (asignados a cada user story) que el equipo completa durante la iteración.
• Se cuentan los story poins de las Users Storys que están completas, no parcialmente completas.
• La Velocidad corrige los errores de estimación.

Propuestas de estimación:

Poker estimation
• Combina opinión de experto, analogía y desegregación.
• Participantes en “planning poker” son desarrolladores
• “Las personas más competentes en resolver una tarea deben ser quienes las estiman”

Planning Poker: La Herramienta Colaborativa

Combina la opinión de expertos, evita el sesgo de anclaje y fuerza las conversaciones técnicas más valiosas precisamente cuando las estimaciones divergen.



NotebookLM

Poker Planning



Prerrequisitos:

- Lista de features/stories a ser estimadas
- Cada estimador tiene un mazo de cartas.

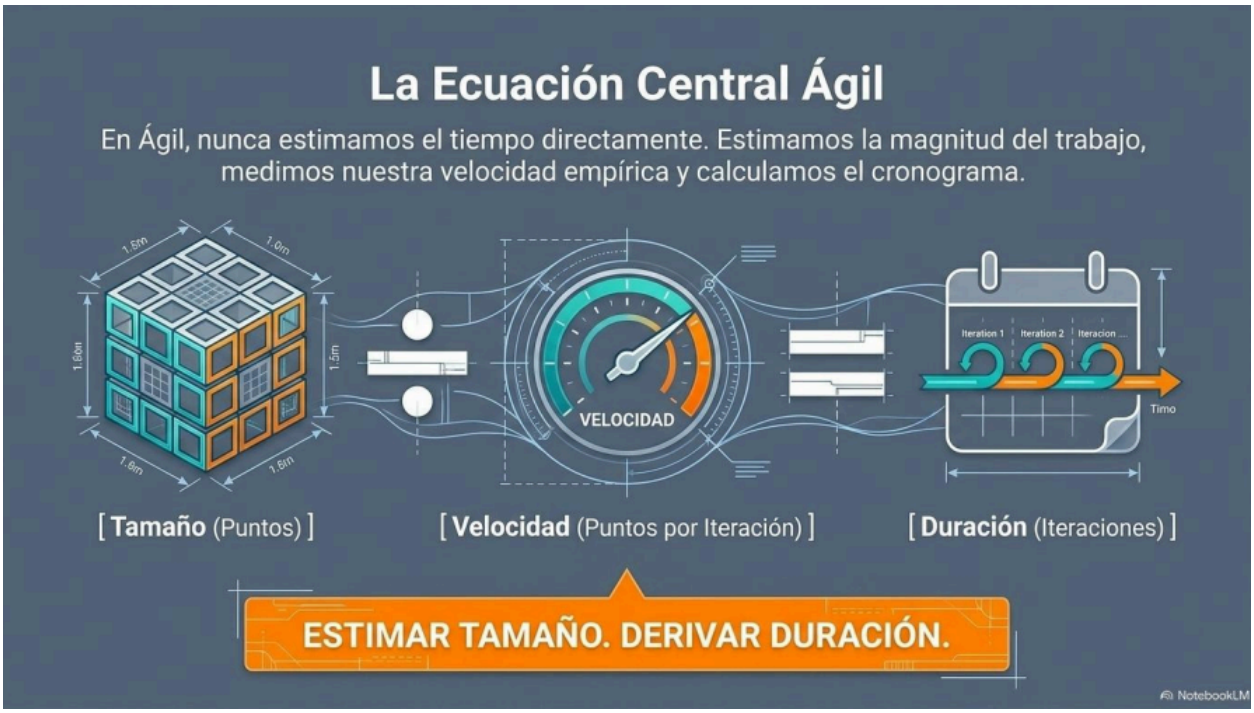
Pasos:

1. Determine la **base story** (la canónica) que será usada para comprar con las otras stories. Digamos, Story Z.
   - 1.1 La story a ser estimada se lee a todo el equipo.
   - 1.2 Los estimadores discuten la story, haciendo preguntas al product owner (las que se necesiten).
   - 1.3 Cada estimador selecciona una carta y pone la carta boca abajo en la mesa.
   - 1.4 Cuando todos pusieron las cartas, las mismas se exponen al mismo tiempo.
   - 1.5 Si todos los estimadores selecciona el mismo valor, ese es el estimado. Sino, los estimadores discuten sus resultados, poniendo especial atención en los más altos y los más bajos. Después de la charla, GOTO to 1.3.
2. Se toma la próxima story, se discute con el product owner.
3. Cada estimador asigna a la story un valor por comparación contra la base story. “Cuan grande/pequeña, compleja, riesgosa es esta story comparada con Story Z?”. GOTO to 1.3

28


Árbol de Decisión: ¿Cuándo debemos Re-estimar?
Re-estimar una tarea solo porque tomó más tiempo del esperado arruina la línea base matemática del proyecto. Dejá que la Velocidad absorba el impacto.
¿Cambió el tamaño relativo de la historia?
SÍ
NO
(Ej: No entendimos la tecnología inicial)
(Ej: Simplemente fuimos más lentos en este sprint, o hubo interrupciones)
RE-ESTIMAR. (Aprender del error y ajustar la base)
NO RE-ESTIMAR


Negociá el Compromiso, nunca la Estimación.
Equipo Técnico
Dueños absolutos de la Estimación.
COMPROMISO
Negocio
Dueños absolutos del Objetivo.
Una estimación es un cálculo basado en datos; negociarla es tan inútil como negociar la temperatura del sol.
Lo que SÍ se negocia es el Compromiso: ajustar el alcance (scope), mover la fecha, o aceptar más riesgo de negocio.



GESTION DE PRODUCTOS



Describe el ciclo de vida completo del desarrollo de un producto de software desde una perspectiva centrada en el valor de negocio y la experiencia del usuario

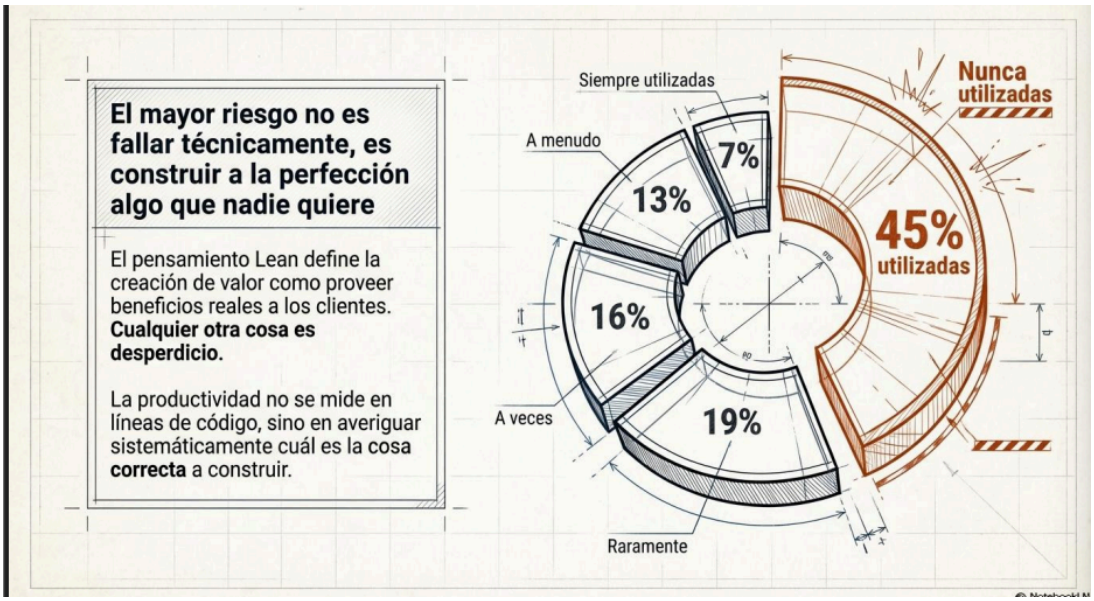
Validacion de hipotesis: Fase de investigacion para saber si existe un problema real que valga la pena resolver y solucion propuesta en viable para el mercado

Definición de alcance: Planificación, diseño de arquitectura y definicion de caracterisiticas (mvp - Producto Minimo Variable) para delimitar que se va a construir concretamente para evitar el desperdicio de recursos y alinear las expectativas del equipo

Diseño y desarrollo: Creación de interfaz y codificación del sistema, para transformar los planos y bocetos en componantes de software estructurados y funcionales

Implementación: Es la integracion, despliegue e infraestructura tecnica, pone el producto en produccion dentro de un entorno estable, seguro y lito para ser utilizado

Crecimiento y experiencia del usuario: Interaccion real del usuario con la plataforma ya pulida, se busca lograr optimizar la experiencia .

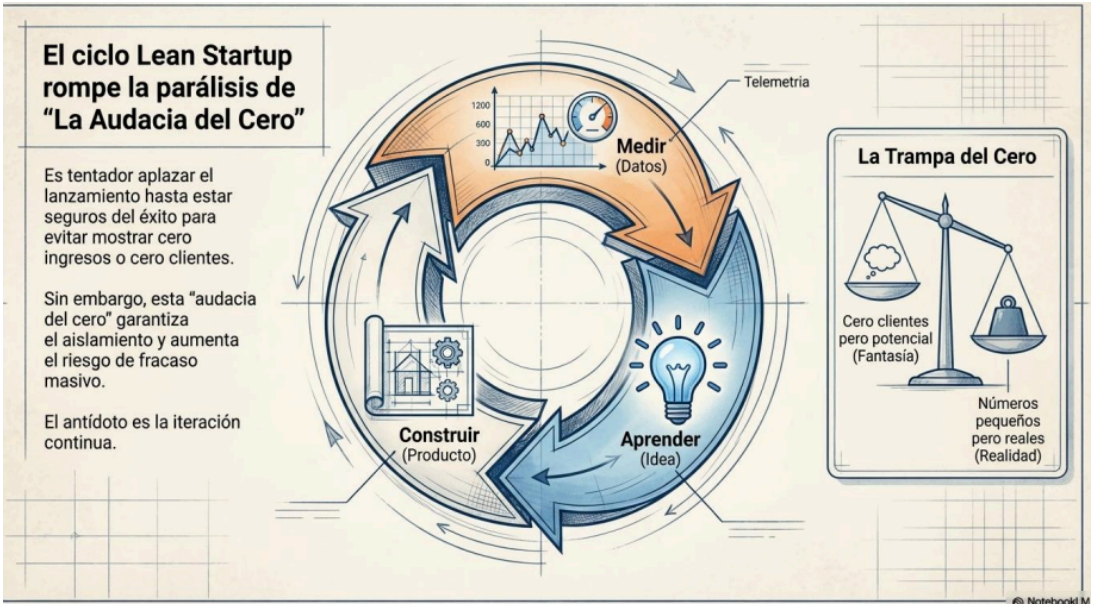


Pensamiento Lean es una filosofia de trabajo que busca maximizar el calor entregado al cliente reduciendo al minimo el desperdicio "El mayor riesgo no es fallar técnicamente, es construir a la perfección algo que nadie quiere."

Principios clave del Pensamiento Lean en desarrollo de software:

1. **Definición de Valor:** El valor se define exclusivamente como aquello que provee beneficios reales a los usuarios finales. Cualquier otra actividad, funcionalidad innecesaria o proceso burocrático se considera **desperdicio** (*muda*).
2. **Eliminar el desperdicio:** En el gráfico circular se muestra una estadística famosa en ingeniería de software: el **45% de las funcionalidades desarrolladas nunca se utilizan**, y solo un **7% se usa siempre**. El pensamiento Lean combate esto enfocándose solo en lo esencial.
3. **Medición de la productividad:** En lugar de medir el progreso en cantidad de tareas terminadas o líneas de código escritas, la productividad se mide en la capacidad de **descubrir e iterar sistemáticamente sobre qué es lo correcto para construir**.
4. **Ciclo de retroalimentación rápido (Crear - Medir - Aprender):** En lugar de pasar años o meses construyendo un producto complejo sin probarlo, se lanzan iteraciones pequeñas (*Producto Mínimo Viable* o MVP) para validar hipótesis directamente con usuarios reales y ajustar la marcha.
5. **Aprender constantemente:** La toma de decisiones se basa en evidencia cuantitativa y cualitativa (datos de uso real) en lugar de suposiciones o intuiciones iniciales.





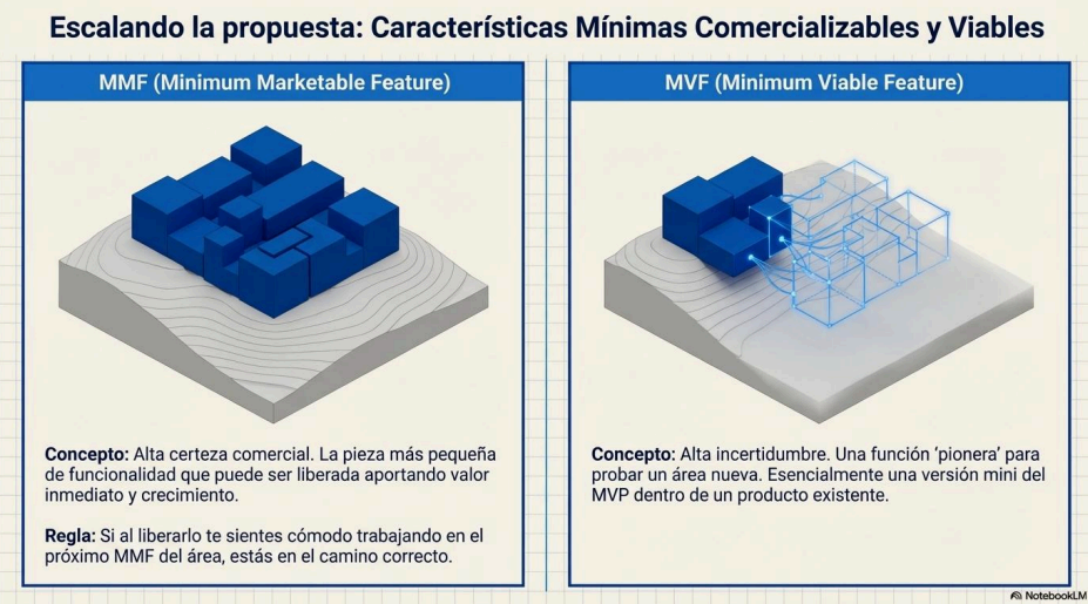
El **ciclo Lean Startup** es la metodología iterativa fundamental para crear productos y negocios con el menor riesgo posible, basada en el bucle continuo de **Construir → Medir → Aprender**.

Las 3 fases del Ciclo Lean Startup:

1. **Construir (Producto):**
   ◦ Se crea una versión mínima del producto (Producto Mínimo Viable o MVP) con los recursos justos para probar una hipótesis concreta.
   ◦ El objetivo no es la perfección técnica, sino poner algo funcional frente a usuarios reales lo antes posible.
2. **Medir (Datos):**
   ◦ Se evalúa cómo reaccionan los usuarios mediante telemetría, métricas y datos cuantitativos/cualitativos reales.
   ◦ Se contrapone la "fantasía" de estimaciones teóricas contra la "realidad" de números pequeños pero reales obtenida en el mercado.
3. **Aprender (Idea):**
   ◦ Con base en los datos recopilados, se decide si la idea de negocio/funcionalidad es válida.
   ◦ Aquí se toma la decisión clave: **pivotar** (cambiar la estrategia/dirección) o **persevarar** (optimizar lo construido).

Un MVP es un experimento diseñado para confrontar la realidad lo antes posible. Requiere ser simple y evitar la sobre-construcción. En caso de duda, simplifique. Un MVP no es solo una versión "a medio hacer" del sistema; su propósito es aportar una dosis de realidad al proceso de desarrollo, enfocado en la **Propuesta de Valor Única (UVP)** y lo mínimo indispensable (*table stakes*) para ser funcional y utilizable Un MVP efectivo no se trata de recortar calidad, sino de **recortar alcance** (*scope*) manteniendo la calidad integral en lo que se decide entregar

UVP = Propuesta de valor unica, representa lo que hace que tu producto o servicio sea unico en el mercado



MMF(caracteristica minima comercializable):

Aplica cuando hay **alta certeza comercial**. Es la pieza de funcionalidad más pequeña que se puede liberar al mercado aportando valor inmediato y generando crecimiento directo.
Regla de oro: Si al liberarla te sientes cómodo pasando inmediatamente a trabajar en el siguiente MMF del área, vas por el camino correcto

MVF(CARACTERISITCA MINIMA VIABLE): Aplica cuando hay **alta incertidumbre**. Es una función "pionera" diseñada para explorar y probar un área nueva o desconocida dentro del sistema.

Relación con el MVP: Es esencialmente una versión en miniatura de un **MVP dentro de un producto que ya existe** para validar hipótesis con usuarios antes de construir toda la infraestructura compleja (representada en líneas transparentes).



MLP (producto minimo adorable): un enfoque que va **más allá de la simple viabilidad** técnica del MVP tradicional para enfocarse en enamorar al usuario desde la primera versión.

- **Reputación de Marca:** En empresas ya consolidadas, un nuevo lanzamiento debe demostrar cuidado genuino por el usuario. Esto evita que el producto parezca un intento frío de lucro o un desarrollo "a medio hacer" sobre la base de clientes actual.
- **Destacar en el Mercado:** Hoy en día un simple MVP no siempre basta. Para competir en mercados maduros o contra rivales bien financiados, la primera versión del producto debe capturar la imaginación y la atención del usuario de inmediato.
- **Bucle de Retroalimentación Leal:** Al evocar emociones positivas desde el día uno, no solo consigues probadores, sino usuarios promotores que están profundamente comprometidos con la evolución del producto y aportan *feedback* constante y confiable.

Matriz Diagnóstica: El espectro de los Productos Mínimos

	MVP (Viable)	MMP (Comercializable)	MLP (Adorable)
Objetivo Principal	Validar concepto de forma económica	Atender a base amplia con funcionalidad clave	Crear conexión emocional e interacción profunda
Enfoque de Diseño	Investigar y empatizar	Construir y refinar / Código y testing	Diseñar para el deleite / Impacto emocional
Beneficio Principal	Informar el alcance del producto	Asegurar la calidad y salida al mercado	Aumentar la lealtad y el amor del usuario
Ejemplo Típico	Entrevistas y videos	Lanzamiento Beta	App con animaciones UI divertidas y personalizadas

COMPONENTES DE PROYECTO DE SOFTWARE

El proceso de Software



- Conjunto estructurado de actividades para desarrollar un sistema de software
- Estas actividades varían dependiendo de la organización y el tipo de sistema que debe desarrollarse.
- Debe ser explícitamente modelado si va a ser administrado.

2

Proceso: La secuencia de pasos ejecutados para un propósito dado (IEEE)
Proceso de Software: Un conjunto de actividades, métodos, prácticas, y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados (Sw-CMM)

PROCESOS EMPIRICOS:

Asume procesos complicados con variables cambiantes. Cuando se repite el proceso, se pueden llegar a obtener resultados diferentes.
La administración y control se realiza través de inspecciones frecuentes y adaptaciones
Son procesos que trabajan bien con procesos creativos y complejos.

- Los **Pilares del Empirismo** (Transparencia, Inspección y Adaptación) constituyen el fundamento teórico sobre el cual se construyen las metodologías ágiles y marcos de trabajo como Scrum.
  1 **. Transparencia**
  Para que sea posible evaluar el progreso real de un proyecto, todos los aspectos significativos del proceso deben ser **visibles y entendibles** tanto para quienes realizan el trabajo como para quienes reciben los resultados.
  • **En qué consiste:** No debe existir ocultamiento de información, problemas o avances. La transparencia requiere que el equipo comparta un lenguaje común y estándares definidos.
  • **Ejemplos prácticos en el desarrollo de software:**
  ◦ **Definición de "Hecho" (*Definition of Done - DoD*):** Un entendimiento unánime entre todos los miembros de qué significa exactamente que un requerimiento o Historia de Usuario esté terminado.
  ◦ **Visibilidad de artefactos:** Un *Product Backlog* claro, priorizado y accesible para todo el equipo y stakeholders.
  ◦ **Sinceridad sobre impedimentos:** Traer a la luz bloqueos técnicos o retrasos de manera inmediata.
  2. Inspección
  Los usuarios y el equipo de desarrollo deben **evaluar con frecuencia** los artefactos de software construidos y el progreso del proyecto para detectar variaciones o desviaciones no deseadas con respecto al objetivo.
  • **En qué consiste:** La inspección no es un control o auditoría externa punitiva,

sino una revisión interna periódica para asegurar la calidad y alineación técnica.

- **Ejemplos prácticos en el desarrollo de software:**
  - **Reunión Diaria (*Daily Scrum*):** Inspección diaria del avance del equipo hacia el objetivo de la iteración.
  - **Revisión del Sprint (*Sprint Review*):** Inspección directa sobre el incremento de software funcional entregado al final de una iteración.
  - **Demostraciones y pruebas continuas:** Inspección del código mediante pruebas automatizadas e integración continua.

3. Adaptación

Si durante la fase de inspección se determina que uno o más aspectos del proceso o del producto se están desviando de los límites aceptables, **el proceso, el producto o los planes deben ajustarse lo antes posible** para minimizar desviaciones futuras.

- **En qué consiste:** Inspeccionar sin adaptar carece de sentido en el empirismo. La adaptación es la capacidad real del equipo para responder al cambio de forma ágil.
- **Ejemplos prácticos en el desarrollo de software:**
  - **Retrospectiva del Sprint (*Sprint Retrospective*):** Ajuste de los procesos internos de trabajo, herramientas o dinámicas del equipo al finalizar una iteración.
  - **Repriorización del Backlog:** Modificar prioridades o reemplazar requerimientos en la siguiente iteración tras recibir feedback del cliente.
  - **Pivote técnico:** Cambiar una arquitectura o biblioteca si la inspección técnica demostró que la solución actual no escala.

PROCESOS DEFINIDOS:

Un **Proceso Definido** (o Modelo Prescriptivo) es un enfoque de desarrollo donde todas las actividades, entradas, salidas y procedimientos están completamente especificados, estandarizados y planificados de antemano.

CICLO DE VIDA DE PROYECTOS DE SOFTWARE

Un ciclo de vida es una **representación formal de un proceso** que grafica y describe cómo se desarrolla el software desde una perspectiva o marco metodológico particular.

Las fases del proceso: Las etapas o actividades necesarias para construir el sistema

El orden de ejecución: La secuencia en la que se deben llevar a cabo esas fases

TIPOS DE CICLO DE VIDA:

Secuencial

• **En qué consiste:** Las fases se ejecutan una tras otra de manera lineal (por ejemplo, el modelo en Cascada).
• **Características:** Una etapa debe completarse y aprobarse completamente antes de comenzar la siguiente. Es idóneo para proyectos con requerimientos muy claros y estables.

Iterativo

• **En qué consiste:** El proyecto se divide en bloques de trabajo más pequeños (*iteraciones* o *sprints*).
• **Características:** En cada ciclo se realiza un mini desarrollo completo (diseño, código y pruebas) para entregar versiones incrementales del software (como vimos con los enfoques Lean y los MVP). Permite incorporar *feedback* continuo y adaptarse a los cambios.

Recursivo

- **En qué consiste:** Aplica patrones o estructuras de ciclo de vida dentro de sí mismo a diferentes niveles de abstracción o subsistemas.
- **Características:** Un componente amplio del sistema puede tener su propio ciclo de vida interno mientras forma parte de un ciclo de vida mayor a nivel general del proyecto. Se utiliza habitualmente en proyectos complejos o arquitecturas basadas en componentes y microservicios.

¿QUE ES UN PROYECTO DE SOFTWARE?

Un **proyecto de software** es un esfuerzo temporal y planificado que tiene como objetivo diseñar, construir, probar, desplegar y mantener un producto, sistema o aplicación de software para resolver un problema específico o satisfacer una necesidad del usuario o del negocio.

Ciclo de vida (SDLC): Todo proyecto sigue un proceso o modelo estructurado que

define las fases de desarrollo (requerimientos, diseño, codificación, pruebas e implementación) y el orden en que se ejecutan (secuencial, iterativo o recursivo).

Gestión del alcance y valor: Consiste en delimitar qué funcionalidades se van a construir para resolver el problema real del usuario (Propuesta de Valor Única / UVP) evitando el desperdicio de recursos.

Construcción iterativa e incremental: Actualmente, la mayoría de los proyectos aplican filosofías como *Lean* o *Agile*, lanzando versiones mínimas viables (**MVP**) o adorables (**MLP**) para medir resultados con datos reales de usuarios e iterar sobre la marcha.

Variables de control: Todo proyecto de software balancea continuamente cuatro restricciones principales:
1. **Alcance (*Scope*):** Las características y requerimientos a desarrollar.
2. **Tiempo:** Los plazos de entrega y cronogramas.
3. **Costo / Recursos:** El presupuesto y el equipo humano/técnico asignado.
4. **Calidad:** La fiabilidad, usabilidad y mantenibilidad del sistema resultante.

Los proyectos son de duración limitada, son temporarios, cuando alcanza el/los objetivo/s, el proyecto termina.

NO SILVER BULLET

No Silver Bullet — Essence and Accident in Software Engineering (*No hay bala de plata: Esencia y accidente en la ingeniería de software*) es uno de los ensayos más célebres de la informática y la ingeniería de software, escrito por **Frederick P. Brooks Jr.** en 1986.

1. Tesis principal del Paper

El título hace referencia a la leyenda folclórica de que solo una **bala de plata** puede matar a un hombre lobo u otro monstruo. Brooks utiliza esta metáfora para advertir que los proyectos de software suelen convertirse en "monstruos incontrolables" de costos y plazos.

Su tesis central establece que:

> **No existe ningún desarrollo individual, ya sea en tecnología o técnica de gestión, que por sí solo prometa una mejora de un orden de magnitud (10x) en productividad, confiabilidad o simplicidad en el plazo de una década.**

2. Dificultades Esenciales vs. Dificultades Accidentales

Brooks clasifica los problemas del desarrollo de software en dos categorías fundamentales (inspirado en la filosofía aristotélica):

1. **Dificultades Accidentales (*Accidental Complexity*):**
    - Son las dificultades asociadas al **proceso de producción** o a las herramientas empleadas.
    - Ejemplos: las limitaciones del hardware, sintaxis complejas de lenguajes de bajo nivel, gestión manual de memoria o tiempos de compilación lentos.
    - *Diagnóstico:* Gran parte de las innovaciones históricas (como los lenguajes de alto nivel o los entornos interactivos) resolvieron problemas accidentales. Sin embargo, a medida que la complejidad accidental disminuye a niveles muy bajos, eliminarla por completo ya no genera saltos gigantescos de productividad global.
2. **Dificultades Esenciales (*Essential Complexity*):**
    - Son los problemas **inherentes a la naturaleza misma del software**.
    - Consisten en la conceptualización, especificación, diseño y verificación de estructuras lógicas abstractas y complejas.
    - *Diagnóstico:* La parte difícil de construir software no es escribir el código, sino decidir con precisión qué se quiere decir y cómo modelar la realidad.

3. Las 4 Propiedades Esenciales del Software

Brooks argumenta que el software es inherentemente difícil debido a cuatro atributos de su esencia:

- **Complejidad (*Complexity*):** Ninguna parte de un programa de software es idéntica a otra a nivel conceptual. A medida que el sistema crece, el número de estados y combinaciones posibles aumenta de forma no lineal, dificultando la comprensión, la comunicación dentro del equipo y la integridad conceptual.

- **Conformidad (*Conformity*):** El software no sigue leyes físicas unificadas. Debe adaptarse de manera arbitraria a las complejidades e inconsistencias impuestas por instituciones, regulaciones, sistemas legados e interfaces creadas por humanos.
- **Cambiabilidad (*Changeability*):** El software se ve sometido a una presión constante de cambio porque es maleable y encarna la funcionalidad de los procesos. A diferencia de un producto de hardware o un edificio (que raramente se modifica sustancialmente una vez construido), al software se le exige evolucionar continuamente.
- **Invisibilidad (*Invisibility*):** El software es abstracto e intangible. No posee una representación geométrica natural (como el plano de un edificio o el diagrama de un circuito integrado), lo que impide visualizar fácilmente la totalidad de sus relaciones lógicas.

4. Evaluación de "Falsas Balas de Plata"

Brooks analiza varias tecnologías y enfoques que en su momento se promocionaron como soluciones revolucionarias:

- **Programación Orientada a Objetos (POO) y Lenguajes como Ada:** Aunque mejoraron la modularidad y la abstracción de datos, no resolvieron la complejidad esencial subyacente.
- **Inteligencia Artificial y Sistemas Expertos:** Identificó que eran útiles para dominios muy acotados, pero no para solucionar el problema conceptual general del software.
- **Programación Gráfica / Verificación Formal:** Aportan valor en la representación o comprobación matemática, pero no eliminan la dificultad de obtener requisitos correctos e integrales.

5. Ataques Prometedores a la Esencia (Soluciones Reales)

Dado que no existe una varita mágica, Brooks sugiere enfocarse en avances conceptuales y metodológicos:

1. **Comprar en lugar de construir (*Buy vs. Build*):** La forma más directa de no asumir el costo de desarrollo de software es reutilizar o comprar paquetes comerciales preexistentes.

2. **Refinamiento de Requisitos y Prototipado Rápido:** Como los clientes rara vez saben exactamente lo que necesitan hasta que interactúan con un sistema, es crucial iterar mediante prototipos rápidos para descubrir los requisitos reales.
3. **Desarrollo Incremental (*Grow vs. Build*):** El software no debe construirse como un bloque monolítico rígido, sino "cultivarse" e incrementarse de forma progresiva a partir de un sistema funcional básico (antecedente directo de las metodologías ágiles).
4. **Cultivar Grandes Diseñadores:** La construcción de software es un proceso profundamente creativo. Las organizaciones deben dedicar esfuerzos a identificar, capacitar y retener a diseñadores de software talentosos con visión de arquitectura e integridad conceptual.

6. Importancia y Vigencia Actual

Incluso hoy, con el auge de tecnologías como la Inteligencia Artificial Generativa y asistentes de código, el principio de **"No Silver Bullet"** se mantiene vigente: las herramientas avanzadas aceleran la escritura de código (reduciendo la fricción o complejidad accidental), pero la **complejidad esencial** —determinar la arquitectura adecuada, comprender el problema del negocio, validar los requisitos y garantizar la coherencia del sistema— sigue dependiendo fundamentalmente de la capacidad de análisis y pensamiento crítico de las personas.

CRISIS DEL SOFTWARE

La **Crisis del Software** es el contexto histórico y metodológico que dio origen a la Ingeniería de Software como disciplina.

- **Origen histórico:** Término acuñado a finales de los años 60, popularizado en la primera conferencia de la OTAN sobre Ingeniería de Software en **Garmisch (Alemania, 1968)**.
- **El problema de fondo:** Al aumentar la potencia del hardware, las demandas de software se volvieron mucho más complejas. La industria no tenía métodos ni procesos estructurados para construir programas a gran escala.
- **Características principales de la Crisis:**
    1. **Desviaciones presupuestarias:** Los proyectos costaban mucho más de lo planificado.

2. **Retrasos crónicos:** Nunca se entregaban a tiempo.
3. **Baja calidad e inestabilidad:** Software con exceso de fallos (*bugs*) e inseguro.
4. **Inmantenibilidad:** El código era casi imposible de modificar o escalar una vez construido.
5. **Incapacidad de gestionar la complejidad:** El software se desarrollaba como una tarea puramente artesanal e individual, sin rigor ingenieril.

Disciplinas del Ciclo de Vida del Producto (Técnicas)

- **Ingeniería de Requerimientos (*Software Requirements*):**
  - Descubrimiento, análisis, especificación (ERS, Historias de Usuario) y validación de lo que el sistema debe hacer y las restricciones que debe cumplir.
- **Diseño de Software (*Software Design*):**
  - Definición de la arquitectura del sistema, estructura de componentes, interfaces, modelos de datos y patrones de diseño antes de escribir código.
- **Construcción de Software (*Software Construction*):**
  - Creación detallada del software mediante la combinación de programación (codificación), depuración (*debugging*), pruebas unitarias e integración de componentes.
- **Pruebas de Software (*Software Testing*):**
  - Verificación y validación del sistema para evaluar si cumple con los requerimientos, detectar fallos (*bugs*) y asegurar su comportamiento esperado.
- **Mantenimiento de Software (*Software Maintenance*):**
  - Actividades posteriores a la entrega del producto para corregir errores, adaptar el sistema a nuevos entornos u optimizar su rendimiento y

funcionalidad.

Disciplinas de Apoyo y Gestión de Procesos:

- **Gestión de la Configuración del Software (*Software Configuration Management - SCM*):**
  - Identificación, control de versiones, trazabilidad y reporte de cambios en todos los artefactos del software (código, documentos, diagramas) a lo largo del ciclo de vida.
- **Gestión de Proyectos de Software (*Software Engineering Management*):**
  - Planificación, seguimiento, estimación de costos y tiempos, gestión de riesgos, asignación de recursos y liderazgo de equipos de desarrollo.
- **Procesos de Ingeniería de Software (*Software Engineering Process*):**
  - Definición, implementación, medición y mejora continua de los modelos de proceso y metodologías de trabajo (ej. Scrum, Cascada, PUD).
- **Aseguramiento de la Calidad de Software (*Software Quality Assurance - SQA*):**
  - Establecimiento de estándares, auditorías y métricas para garantizar que tanto el producto final como el proceso de desarrollo cumplan los niveles de calidad requeridos.
- **Herramientas y Métodos de la Ingeniería de Software (*Software Engineering Tools and Methods*):**
  - Uso de entornos de desarrollo, herramientas CASE, lenguajes de modelado (UML), integración continua y marcos de trabajo técnicos.
- **Economía de la Ingeniería de Software (*Software Engineering Economics*):**
  - Análisis de decisiones técnicas ponderando variables financieras, como retorno de inversión (ROI), análisis costo-beneficio y evaluación de alternativas (*Build vs. Buy*).

Ventajas y desventajas de seleccion de ciclos de vida

A. Ciclo de Vida Secuencial / Tradicional (Cascada / Modelo V)
Las fases (Requerimientos, Diseño, Implementación, Pruebas, Mantenimiento) se ejecutan de forma lineal y en orden estricto; no se avanza a la siguiente etapa hasta terminar completamente la anterior.

- **Ventajas:**
  - **Simplicidad y claridad:** Es fácil de entender, implementar y gestionar.
  - **Hitos e informes definidos:** Cada fase produce artefactos y documentación clara.
  - **Facilidad para estimar:** Si los requerimientos son fijos, es más sencillo estimar costos y tiempos al inicio.
- **Desventajas:**
  - **Rigidez ante el cambio:** Modificar requerimientos en fases avanzadas es extremadamente costoso.
  - **Pruebas tardías:** La integración y validación se realizan al final; los fallos graves de diseño se descubren tarde.
  - **Efecto "Bloque de Hielo":** El cliente no ve un producto software funcional hasta las fases finales del proyecto.

B. Ciclo de Vida Iterativo e Incremental
El proyecto se divide en pequeñas entregas (incrementos). En cada iteración se realiza un ciclo completo de desarrollo para agregar nuevas funcionalidades sobre una base previamente construida.

- **Ventajas:**
  - **Entrega temprana de valor:** El cliente obtiene versiones funcionales desde las primeras iteraciones.
  - **Mayor flexibilidad:** Permite acomodar cambios de requisitos entre iteraciones con menor impacto.
  - **Reducción de riesgo:** Los problemas de integración o arquitectura se identifican en etapas tempranas.
- **Desventajas:**
  - **Complejidad arquitectónica:** Requiere una arquitectura sólida e inicial para permitir agregar piezas sin romper lo existente.

◦ **Riesgo de "deuda técnica":** Sin una refactorización constante, el código puede degradarse rápidamente.
◦ **Dificultad de estimación total:** Si el alcance final varía, el costo total puede ser indeterminado al inicio.

C. Ciclo de Vida Evolutivo / Basado en Prototipos

Se enfoca en la construcción rápida de prototipos interactivos para explorar y definir los requerimientos junto con el cliente antes de construir la versión definitiva.

• **Ventajas:**
◦ **Aclaración de requerimientos:** Elimina la ambigüedad y ayuda al cliente a visualizar lo que necesita.
◦ **Alta satisfacción del usuario:** Involucra activamente a los usuarios finales en el diseño.
◦ **Disminución de errores de usabilidad y funcionalidad.**
• **Desventajas:**
◦ **Falsas expectativas:** El cliente puede pensar que, porque el prototipo se ve bien, el sistema completo está "casi terminado".
◦ **Código descartable vs. código de producción:** Existe la tentación de emparchar el prototipo y lanzarlo a producción sin la calidad/arquitectura necesaria.

D. Ciclo de Vida Orientado a Riesgos (Modelo en Espiral)

Combina la naturaleza iterativa con la evaluación formal de riesgos en cada ciclo (Planificación, Análisis de Riesgos, Ingeniería/Desarrollo, Evaluación del Cliente).

• **Ventajas:**
◦ **Gestión proactiva de riesgos:** Es el modelo más fuerte para proyectos críticos y de alto nivel de incertidumbre.
◦ **Apto para grandes sistemas:** Permite integrar elementos de otros modelos (prototipado, cascada) dentro de las espirales.
• **Desventajas:**
◦ **Alto costo y complejidad:** Requiere especialistas expertos en análisis de riesgos.
◦ **Gran sobrecarga de gestión:** No es viable para proyectos pequeños o de bajo presupuesto.

E. Ciclo de Vida Ágil (Adaptativo / Scrum / Kanban)

Proceso iterativo en ciclos muy cortos (sprints) con enfoque en la colaboración continua, la adaptación al cambio y la entrega de software funcionando por sobre

la documentación extensiva.

- **Ventajas:**
  - **Máxima adaptabilidad:** El cambio de requisitos se asume como parte natural del proceso.
  - **Feedback continuo:** Transparencia total e interacción directa con el cliente/Product Owner.
  - **Calidad continua:** Pruebas integradas dentro de cada iteración.
- **Desventajas:**
  - **Exige alta disponibilidad del cliente:** Requiere presencia activa del negocio en todo momento.
  - **Pérdida de visión global / Arquitectura:** Sin el debido cuidado, se corre el riesgo de perder la perspectiva general del sistema.
  - **Documentación escasa:** Si no se gestiona bien, puede dificultar el mantenimiento futuro.

2. Criterios para la Selección de un Ciclo de Vida

Para decidir cuál ciclo de vida aplicar, un ingeniero o líder de proyecto analiza variables agrupadas en **5 dimensiones clave**:

1. Requerimientos

- **¿Son estables, conocidos y bien definidos desde el inicio?** → *Secuencial (Cascada / Modelo V).*
- **¿Son poco claros, cambiantes o de alta volatilidad?** → *Ágil / Prototipado / Incremental.*

2. Criticidad del Sistema y Gestión de Riesgos

- **¿Es un sistema crítico (de vida, seguridad nacional o financiero masivo) donde un fallo es inaceptable?** → *Modelo V / Espiral / Proceso Formal Definido.*
- **¿El proyecto involucra alta incertidumbre técnica o de negocio?** → *Espiral / Prototipado (Spikes).*

3. Equipo de Desarrollo y Cliente

- **¿El cliente está disponible diariamente para dar feedback?** → *Ágil.*
- **¿El cliente solo se involucra al inicio (contrato de alcance cerrado) y al final?** → *Secuencial / Cascada.*
- **¿El equipo tiene experiencia con la tecnología y el dominio del problema?**
  - *Sí:* Secuencial o Incremental.
  - *No (tecnología o dominio nuevo):* Prototipado o Ágil para validación temprana.

4. Tamaño y Complejidad del Proyecto

- **Proyectos pequeños/medianos con tiempos acotados:** $\rightarrow$ *Ágil o*

Incremental.
• **Proyectos gigantescos, multisistema y distribuidos entre muchas empresas:** → *Modelos híbridos o Marcos escalados (PUD / Espiral).*
5. Restricciones de Tiempo y Presupuesto (*Time-to-Market*)
• **Si el factor clave es salir al mercado lo antes posible con funcionalidades mínimas (MVP):** → *Incremental / Ágil.*
• **Si hay un presupuesto estricto, fijo y regulado por licitación pública:** → Secuencial / Cascada

MANIFIESTO AGIL

El **Manifiesto Ágil** (*Agile Manifesto*) es una declaración redactada en febrero de 2001 en Snowbird, Utah, por 17 líderes de la industria del software para formalizar un nuevo enfoque de desarrollo frente a los métodos tradicionales y prescriptivos (como Cascada). Se fundamenta en **4 Valores Fundamentales** y **12 Principios**, priorizando la entrega de valor, la adaptabilidad al cambio y las personas sobre los procesos rígidos y la documentación extensiva.

Los 4 Valores Fundamentales del Manifiesto Ágil

Aunque se reconoce el valor de los elementos de la derecha, se valora más los de la izquierda:

1. **Individuos e interacciones** sobre procesos y herramientas.
2. **Software funcionando** sobre documentación exhaustiva.
3. **Colaboración con el cliente** sobre negociación contractual.
4. **Respuesta ante el cambio** sobre el seguimiento de un plan.

Los 12 Principios del Manifiesto Ágil

1. **Satisfacción del cliente:** Nuestra mayor prioridad es satisfacer al cliente mediante la entrega temprana y continua de software con valor.
2. **Aceptación del cambio:** Aceptamos que los requisitos cambien, incluso en etapas tardías del desarrollo. Los procesos ágiles aprovechan el cambio para

proporcionar una ventaja competitiva al cliente.

3. **Entrega frecuente:** Entregamos software funcional frecuentemente, en periodos que van desde un par de semanas a un par de meses, con preferencia por la escala de tiempo más corta.
4. **Trabajo conjunto:** Los responsables de negocio y los desarrolladores deben trabajar juntos de forma cotidiana a lo largo de todo el proyecto.
5. **Individuos motivados:** Construimos proyectos en torno a individuos motivados. Hay que darles el entorno y el apoyo que necesitan, y confiar en ellos para realizar el trabajo.
6. **Conversación cara a cara:** El método más eficiente y efectivo de comunicar información al equipo de desarrollo (y entre sus miembros) es la conversación cara a cara.
7. **Software funcionando como medida de progreso:** El software en funcionamiento es la principal medida de avance del proyecto.
8. **Desarrollo sostenible:** Los procesos ágiles promueven el desarrollo sostenible. Los patrocinadores, desarrolladores y usuarios deben ser capaces de mantener un ritmo constante de forma indefinida.
9. **Excelencia técnica:** La atención continua a la excelencia técnica y al buen diseño mejora la agilidad.
10. **Simplicidad:** La simplicidad —entendida como el arte de maximizar la cantidad de trabajo no realizado— es esencial.
11. **Equipos autoorganizados:** Las mejores arquitecturas, requisitos y diseños emergen de equipos autoorganizados.
12. **Mejora continua y reflexión:** A intervalos regulares, el equipo reflexiona sobre cómo ser más eficaz para, a continuación, ajustar y perfeccionar su comportamiento en consecuencia.

SCRUM

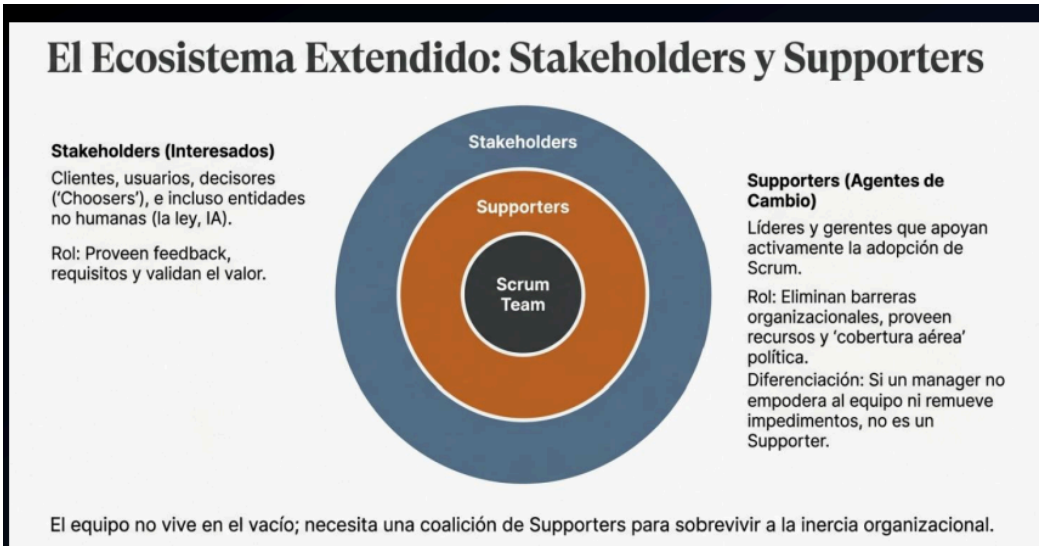
Scrum es un marco de trabajo liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos.

PRINCIPIOS DEL SCRUM

- **Control del Proceso Empírico (*Empirical Process Control*):**
  El trabajo se gestiona en función de la experiencia real y los hechos observados. Se sostiene sobre los 3 pilares clave: **Transparencia** (visibilidad total del estado del proyecto), **Inspección** (evaluación frecuente del producto y proceso) y **Adaptación** (ajuste inmediato ante desviaciones).
- **Autoorganización (*Self-organization*):**
  Los equipos de desarrollo no reciben instrucciones jerárquicas de *"cómo"* realizar el trabajo. Tienen la autonomía, responsabilidad y confianza necesarias para organizarse internamente, evaluar sus capacidades y resolver problemas, lo que incrementa el compromiso y la innovación.
- **Colaboración (*Collaboration*):**
  El desarrollo de software se concibe como un proceso interactivo donde el equipo de desarrollo, el *Product Owner* y las partes interesadas (*stakeholders*) trabajan integradamente mediante tres dimensiones:
  - **Conciencia:** Todos conocen el trabajo del resto.
  - **Articulación:** División coherente de tareas.
  - **Apropiación:** Sentido de pertenencia compartido sobre el resultado final.
- **Priorización Basada en el Valor (*Value-based Prioritization*):**
  Los requerimientos en el *Product Backlog* se ordenan continuamente de acuerdo al valor de negocio que aportan al cliente y al riesgo involucrado. El objetivo es entregar siempre el máximo valor funcional posible en el menor tiempo (*Time-to-Market*).
- **Bloques de Tiempo (*Time-boxing*):**
  Se fijan límites de tiempo estrictos y no negociables para todas las actividades y eventos de Scrum (ej. *Sprints* de 2 a 4 semanas, *Daily* de 15 minutos). Esto evita la postergación de decisiones, previene el desgaste del equipo y ayuda a mantener un ritmo de trabajo sostenible.

- **Desarrollo Iterativo (*Iterative Development*):**
  El software no se entrega de forma monolítica al final del proyecto, sino mediante incrementos funcionales periódicos. Cada iteración permite ajustar el rumbo del producto según los cambios en el mercado, el feedback del usuario o el aprendizaje técnico obtenido.





Rol	Naturaleza	Responsabilidad Principal
Product Owner	Humano	Maximizar el valor a largo plazo; gestionar el Product Backlog y la estrategia del producto.
Product Developer	Humano o Automatizado	Crear incrementos utilizables; adherirse a la Definición de Salida Terminada; autogestionar el plan del Sprint.
Scrum Master	Humano	Agente de cambio; guía la efectividad del equipo y la organización en la adopción de Scrum.
Stakeholder	**Humano/Entidad/IA**	Entidades interesadas o afectadas por el producto (clientes, usuarios, patrocinadores, legisladores).
Supporter	**Stakeholder específico**	Coalición poderosa que elimina impedimentos organizacionales y fomenta un clima para la autogestión.

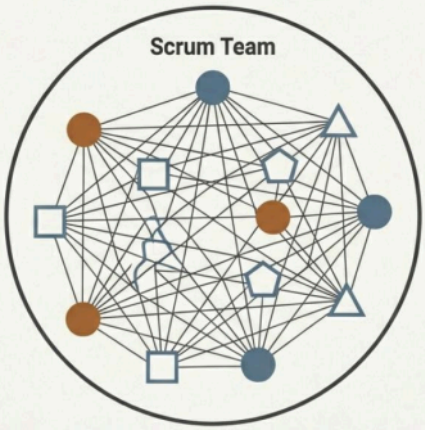


El Scrum Team: Autogestión e Inteligencia Aumentada

Un equipo pequeño, cognitivamente diverso y empoderado que posee todas las habilidades para crear valor.

Autogestión (Self-Managing)

El equipo decide quién hace qué, cómo y cuándo. No hay jerarquías, solo responsabilidades compartidas.

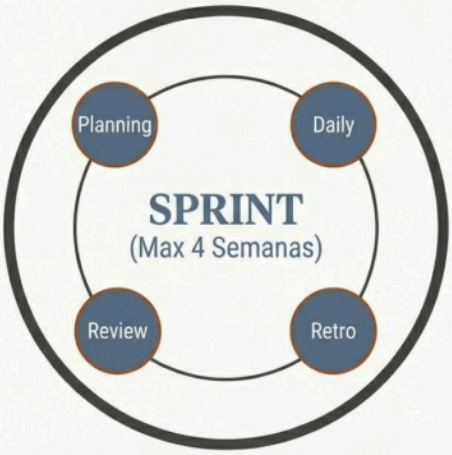


AI como 'Compañero de Equipo'

La Inteligencia Artificial se integra en el descubrimiento y entrega.

- **Human in the loop**: La IA aumenta la capacidad, pero la responsabilidad (accountability) permanece 100% en los humanos.
- **Uso ético**: La IA ayuda a validar hipótesis y reducir sesgos, no a reemplazar el juicio profesional.

El Sprint: El Corazón del Ritmo y la Cadencia



Contenedor de Consistencia

- **Cadencia**: Crea hábito y reduce la complejidad cognitiva.
- **Control de Riesgo**: Sprints más cortos = Menor costo de error. Cada Sprint es una oportunidad para pivotar.

Inmutabilidad del Objetivo

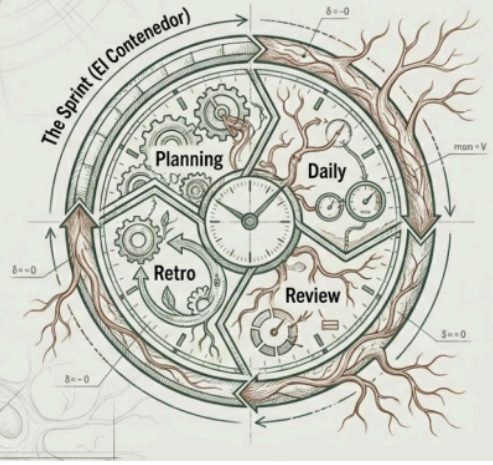
- El Sprint Goal es inamovible.
- El alcance (scope) es negociable entre PO y Developers si ocurre aprendizaje.

No hay "Sprints de Hardening"

La calidad (Output Done) se construye en cada Sprint.

NotebookLM

El Ritmo: Cadencia y Descubrimiento



The Sprint

- Un contenedor “seguro para fallar” (safe-to-fail).
- Estabilidad dentro del caos.
- Frecuencia de aprendizaje > Velocidad.

Sprint Planning

- Definir el Porqué (Sprint Goal).
- Definir el Qué (Forecast) y el Cómo.
- Planificar para terminar temprano permite manejar la sorpresa.

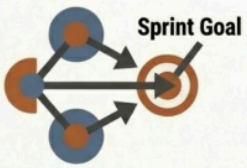
Bucles de Retroalimentación: Inspección Activa



La inspección sin adaptación es desperdicio

Planificación y Sincronización: Inspección Diaria

Sprint Planning (El Diseño Táctico)



Define tres aspectos críticos:

1. **Por Qué**: El Sprint Goal.
2. **Qué**: Items seleccionados del Backlog.
3. **Cómo**: Plan técnico de ejecución.

Nota: El equipo no debe sobrecargarse; planificar holgura (**slack**) para manejar la incertidumbre.

Daily Scrum (Navegación Táctica)



Una replanificación de 15 minutos de los Developers para los Developers.

- **NO ES:** Un reporte de estado para el Scrum Master.
- **ES:** Sincronización táctica.
- **FOCO:** Progreso hacia el Sprint Goal y eliminación de bloqueos inmediatos.

NotebookLM

Revisión y Mejora: Cerrando el Ciclo de Aprendizaje



Sprint Review (Validación de Valor)

- Sesión de trabajo colaborativa, no una presentación.
- Inspección del Incremento real.
- Validación contra la **Definition of Outcome Done**: ¿Resolvimos el problema del usuario?

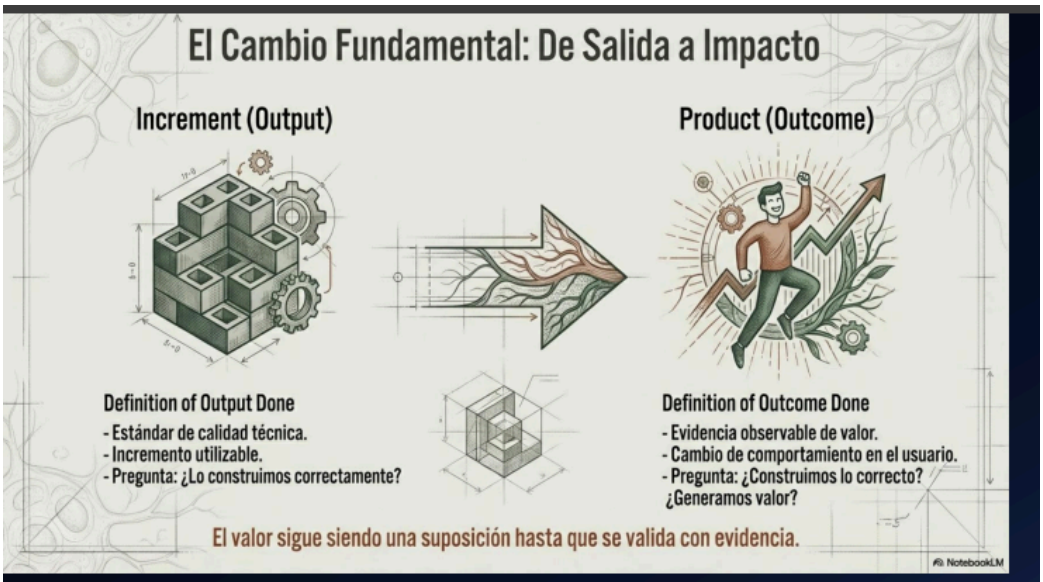


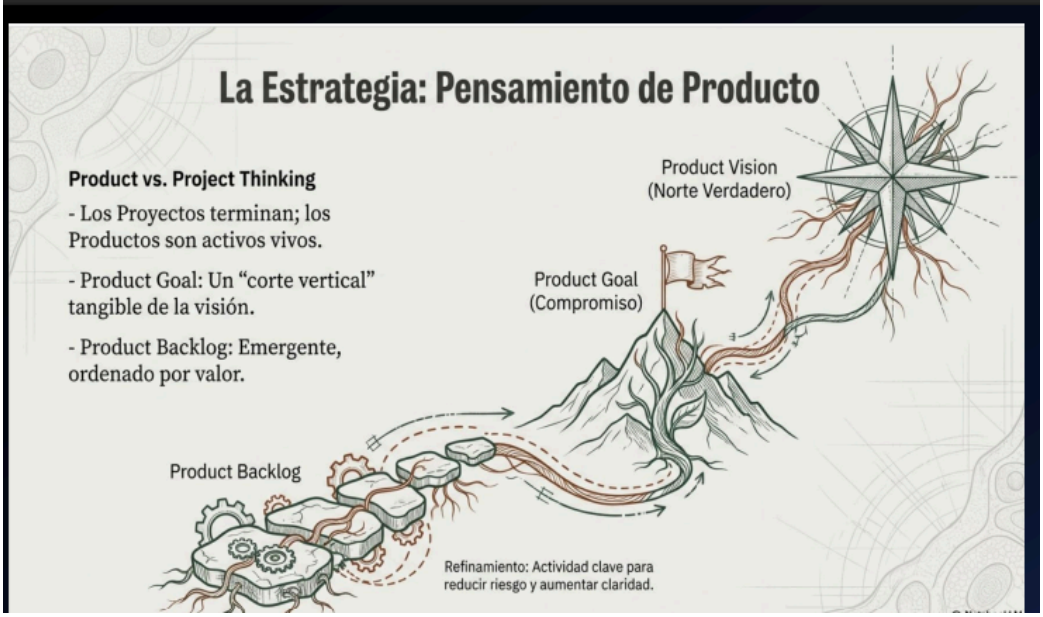
Sprint Retrospective

- Foco en la calidad del equipo, procesos y herramientas.
- Búsqueda de la "Quality Without a Name" (coherencia y vitalidad).

- Búsqueda de la "**Quality Without a Name**" (coherencia y vitalidad).
- Las mejoras deben ejecutarse lo antes posible, incluso dentro del próximo Sprint.

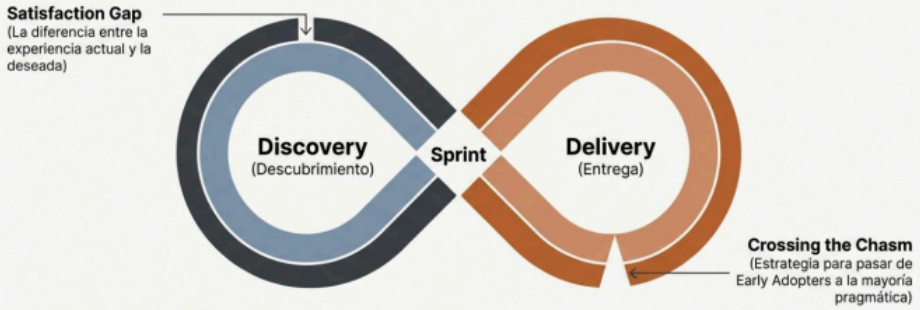

Timebox en Scrum
Timebox
Sprint: 1 mes o menos
Sprint Planning: 8 horas máximo para un Sprint de 1 mes
Daily Meeting: 15 minutos
Sprint Review: 4 horas máximo para un Sprint de 1 mes
Sprint Retrospective: 3 horas máximo para un Sprint de 1 mes
Refinamiento del PB: 10% del tiempo de Sprint


El Cambio Fundamental: De Salida a Impacto
Increment (Output)
Product (Outcome)
Definition of Output Done
- Estándar de calidad técnica.
- Incremento utilizable.
- Pregunta: ¿Lo construimos correctamente?
Definition of Outcome Done
- Evidencia observable de valor.
- Cambio de comportamiento en el usuario.
- Pregunta: ¿Construimos lo correcto? ¿Generamos valor?
El valor sigue siendo una suposición hasta que se valida con evidencia.
NotebookLM


La Estrategia: Pensamiento de Producto
Product vs. Project Thinking
- Los Proyectos terminan; los Productos son activos vivos.
- Product Goal: Un "corte vertical" tangible de la visión.
- Product Backlog: Emergente, ordenado por valor.
Product Vision
(Norte Verdadero)
Product Goal
(Compromiso)
Product Backlog
Refinamiento: Actividad clave para reducir riesgo y aumentar claridad.

Product Thinking: Del Proyecto Temporal al Valor Continuo

Los proyectos terminan; los productos evolucionan. Scrum gestiona el ciclo de vida completo, desde la cuna hasta la tumba.



Discovery & Delivery no son fases separadas. El Product Discovery ocurre dentro del Sprint.

NotebookLM

31

Artefactos y Compromisos: La Estructura de la Transparencia

Product Backlog Lista emergente y ordenada de lo que se necesita.	→	**Product Goal** El objetivo estratégico a mediano plazo (El 'Por qué' estratégico).
Sprint Backlog El plan de los Developers para el Sprint.	→	**Sprint Goal** El objetivo único y coherente del Sprint (El 'Por qué' táctico).
Increment El ladrillo concreto de valor entregado.	→	**Definition of Done** El estándar de calidad requerido para liberar.

La Doble Definición de "Hecho": Output vs. Outcome

Definition of OUTPUT Done
(Calidad Técnica)

Enfoque: ¿Construimos el producto correctamente?

Criterios:
- Código limpio y testado
- Integrado y Documentado
- Cumple estándares

Momento: Debe cumplirse dentro del Sprint para considerar el Incremento.

Definition of OUTCOME Done
(Valor de Negocio)

Enfoque: ¿Construimos el producto correcto?

Criterios:
- Cambio en comportamiento del usuario
- Impacto en el negocio / ROI
- Validación de hipótesis

Momento: Medible a menudo después del lanzamiento (Lagging indicator).

El Output es el requisito previo; el Outcome es el éxito real.

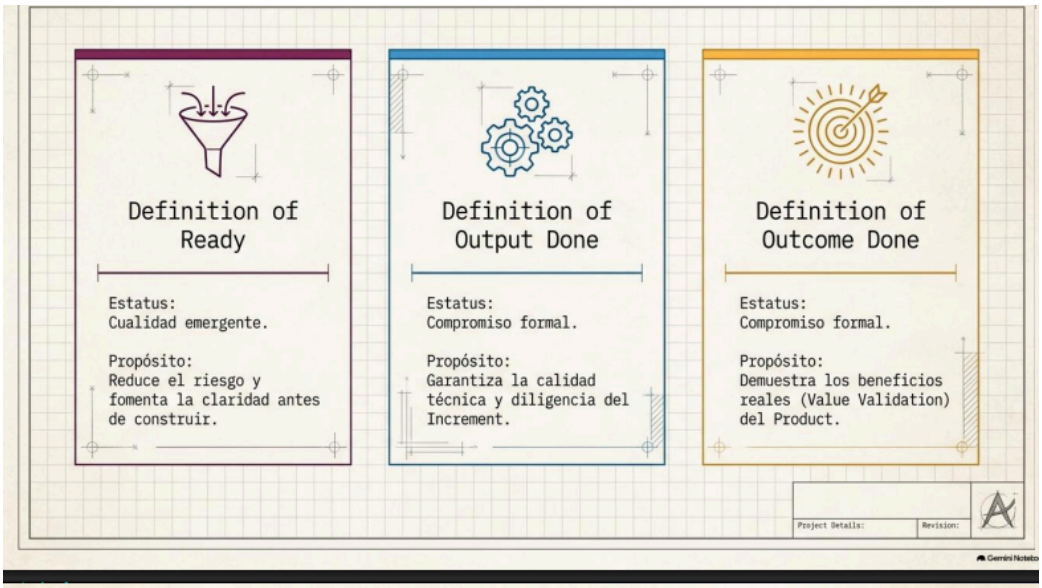
NotebookLM

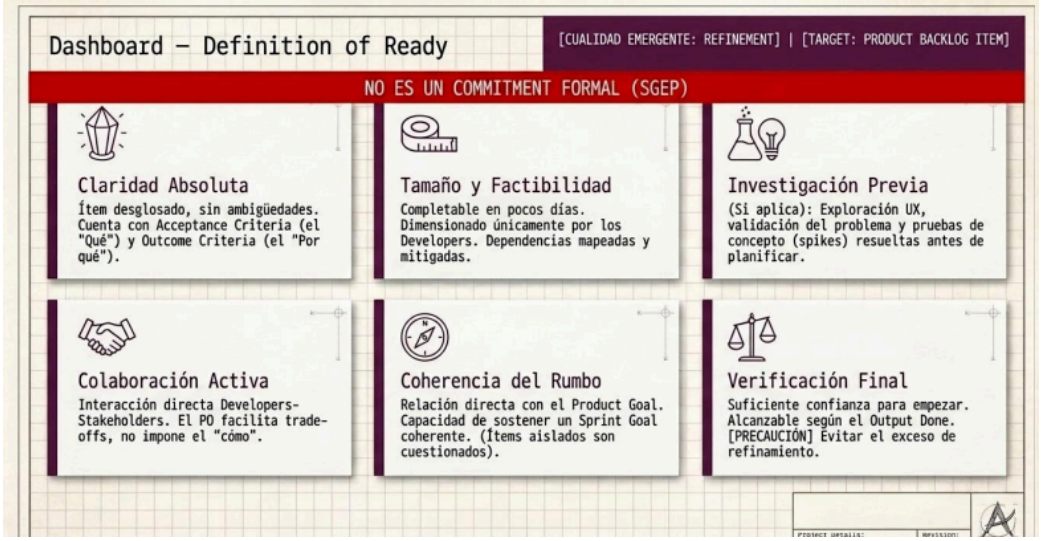
Refinamiento y Backlog: La Claridad Precede a la Velocidad

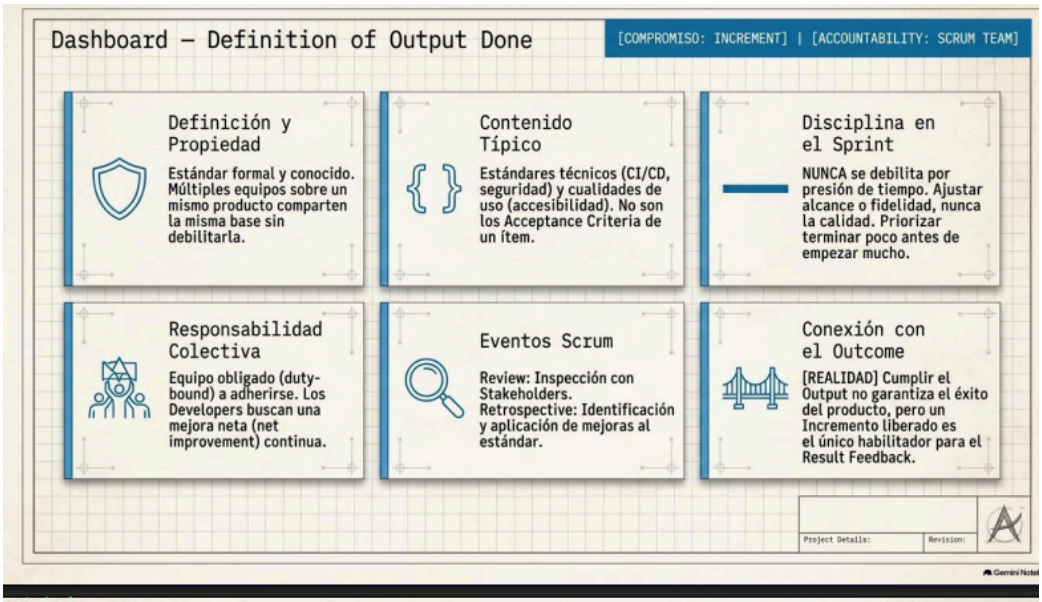
El Product Backlog es un organismo vivo, no un plan fijo.

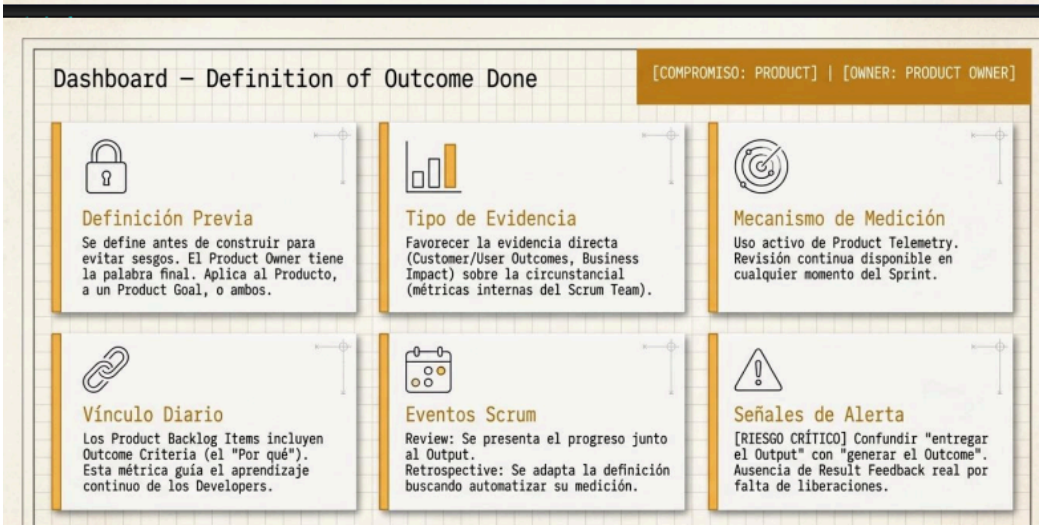


- **Refinamiento Continuo:** Actividad esencial para descomponer ítems y reducir incertidumbre.
- **Aceptando la Incertidumbre:** El refinamiento incluye investigación y validación. "Todo lo que creemos saber puede estar equivocado".
- **Priorización por Valor:** El Product Owner ordena basándose en el valor a largo plazo, no en la urgencia.


Definition of Ready
Estatus:
Cualidad emergente.
Propósito:
Reduce el riesgo y fomenta la claridad antes de construir.
Definition of Output Done
Estatus:
Compromiso formal.
Propósito:
Garantiza la calidad técnica y diligencia del Increment.
Definition of Outcome Done
Estatus:
Compromiso formal.
Propósito:
Demuestra los beneficios reales (Value Validation) del Product.
Project Details:
Revision:


Dashboard – Definition of Ready
[CUALIDAD EMERGENTE: REFINEMENT] | [TARGET: PRODUCT BACKLOG ITEM]
NO ES UN COMMITMENT FORMAL (SGEP)
Claridad Absoluta
Ítem desglosado, sin ambigüedades. Cuenta con Acceptance Criteria (el "Qué") y Outcome Criteria (el "Por qué").
Tamaño y Factibilidad
Completable en pocos días. Dimensionado únicamente por los Developers. Dependencias mapeadas y mitigadas.
Investigación Previa
(Si aplica): Exploración UX, validación del problema y pruebas de concepto (spikes) resueltas antes de planificar.
Colaboración Activa
Interacción directa Developers-Stakeholders. El PO facilita trade-offs, no impone el "cómo".
Coherencia del Rumbo
Relación directa con el Product Goal. Capacidad de sostener un Sprint Goal coherente. (Ítems aislados son cuestionados).
Verificación Final
Suficiente confianza para empezar. Alcanzable según el Output Done. [PRECAUCIÓN] Evitar el exceso de refinamiento.


Dashboard – Definition of Output Done
[COMPROMISO: INCREMENT] | [ACCOUNTABILITY: SCRUM TEAM]
Definición y Propiedad
Estándar formal y conocido. Múltiples equipos sobre un mismo producto comparten la misma base sin debilitarla.
Contenido Típico
Estándares técnicos (CI/CD, seguridad) y cualidades de uso (accesibilidad). No son los Acceptance Criteria de un ítem.
Disciplina en el Sprint
NUNCA se debilita por presión de tiempo. Ajustar alcance o fidelidad, nunca la calidad. Priorizar terminar poco antes de empezar mucho.
Responsabilidad Colectiva
Equipo obligado (duty-bound) a adherirse. Los Developers buscan una mejora neta (net improvement) continua.
Eventos Scrum
Review: Inspección con Stakeholders.
Retrospective: Identificación y aplicación de mejoras al estándar.
Conexión con el Outcome
[REALIDAD] Cumplir el Output no garantiza el éxito del producto, pero un Incremento liberado es el único habilitador para el Result Feedback.
Project Details:
Revision:


Dashboard – Definition of Outcome Done
[COMPROMISO: PRODUCT] | [OWNER: PRODUCT OWNER]
Definición Previa
Se define antes de construir para evitar sesgos. El Product Owner tiene la palabra final. Aplica al Producto, a un Product Goal, o ambos.
Tipo de Evidencia
Favorecer la evidencia directa (Customer/User Outcomes, Business Impact) sobre la circunstancial (métricas internas del Scrum Team).
Mecanismo de Medición
Uso activo de Product Telemetry. Revisión continua disponible en cualquier momento del Sprint.
Vínculo Diario
Los Product Backlog Items incluyen Outcome Criteria (el "Por qué"). Esta métrica guía el aprendizaje continuo de los Developers.
Eventos Scrum
Review: Se presenta el progreso junto al Output.
Retrospective: Se adapta la definición buscando automatizar su medición.
Señales de Alerta
[RIESGO CRÍTICO] Confundir "entregar el Output" con "generar el Outcome". Ausencia de Result Feedback real por falta de liberaciones.

Matriz de Calibración Estratégica

	Definition of Ready	Definition of Output Done	Definition of Outcome Done
Estatus en el SGEP	Cualidad emergente (Refinement).	Compromiso formal.	Compromiso formal.
Artefacto Asociado	Product Backlog Item (PBI).	Increment.	Product.
Responsabilidad y Decisión	Scrum Team colaborativo (Devs estiman).	Developers (Accountability colectiva).	Product Owner (Final say).
Métrica Típica	Días / Capacidad de comprensión.	Tests, Seguridad, Estándares técnicos.	Telemetría, Comportamiento de usuario, ROI.

Project Details: Revision:



HERRAMIENTAS DEL SCRUM:

EL TASKBOARD (tablero de tareas) Su objetivo central es hacer efectivo el pilar de la **Transparencia** en el proceso empírico. Permite que cualquier miembro del equipo o interesado (*stakeholder*) comprenda de un vistazo el estado del proyecto, qué se está construyendo, quién lo está haciendo y qué tareas faltan por completar

El tablero organiza el *Sprint Backlog* dividiendo los elementos en columnas que representan las etapas del flujo de trabajo:

1. **Historias de Usuario / Elementos del Sprint Backlog:** Las funcionalidades o requerimientos comprometidos para el Sprint actual.
2. **Por Hacer (*To Do*):** La lista de tareas individuales identificadas para completar cada Historia de Usuario que aún no han comenzado.
3. **En Proceso (*In Progress*):** Tareas en las que uno o varios desarrolladores están trabajando activamente.
4. **En Verificación / Pruebas (*Testing / Code Review*):** Tareas terminadas que están en proceso de revisión de código, pruebas automatizadas o validación (columna opcional según el equipo).
5. **Hecho (*Done*):** Tareas completamente finalizadas que cumplen con la **Definición de Hecho** (*Definition of Done* - DoD) del equipo.

Sprint Burndown Charts (grafico de trabajo pendiente del sprint)
es una herramienta visual utilizada en Scrum para medir y rastrear la cantidad de trabajo que queda por hacer en el *Sprint Backlog* a lo largo del tiempo que dura la iteración.

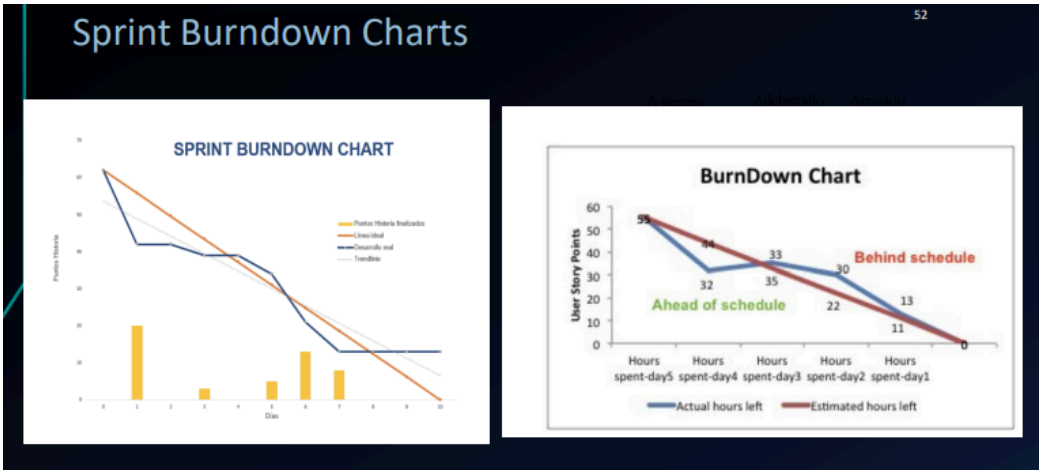
1. Estructura del Gráfico

- **Eje Horizontal (X):** Representa el tiempo, generalmente dividido en los días que dura el Sprint (ej. Día 1 al Día 14).
- **Eje Vertical (Y):** Representa el esfuerzo o trabajo pendiente. Puede medirse en **Horas** (tiempo estimado de las tareas) o en **Story Points** (puntos de historia de las funcionalidades).
- **Línea de Proceso Ideal (*Ideal Line*):** Es una línea diagonal de referencia que va desde el esfuerzo total estimado el día 1, hasta cero el último día del Sprint. Muestra el ritmo constante al que el equipo debería completar el trabajo para terminar exactamente a tiempo.
- **Línea de Progreso Real (*Actual Line*):** Es la curva trazada día a día basándose en el trabajo que verdaderamente falta completar. Se actualiza habitualmente antes o durante la *Daily Scrum*.

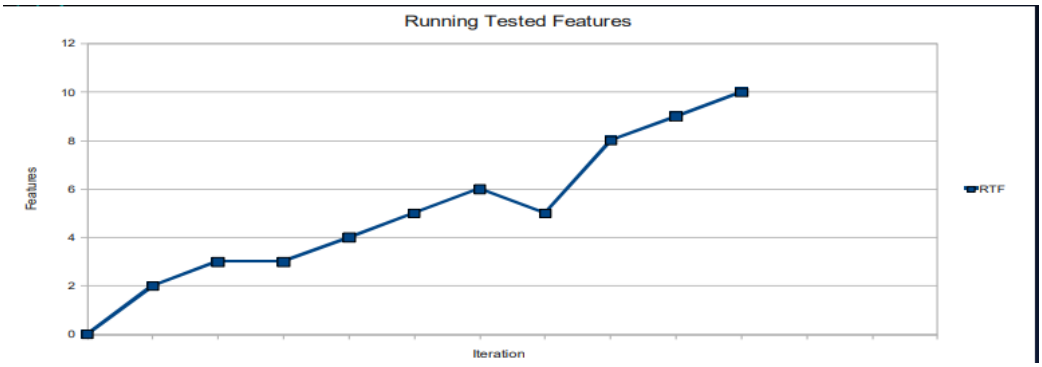
2. Propósito y Beneficios

Su función central es hacer efectivo el principio de control empírico, facilitando la **Inspección y Adaptación**. Permite al equipo de desarrollo responder rápidamente a la pregunta: *"¿Vamos a alcanzar el objetivo del Sprint (Sprint Goal) a este ritmo?"*.

Además, fomenta la **Transparencia**, ya que cualquiera que mire el gráfico puede entender instantáneamente la salud y el avance del Sprint sin necesidad de leer informes extensos.



Running Tested Features (RTF): **RTF** mide en cualquier momento del proyecto la cantidad de funcionalidades que están integradas en el sistema en ejecución y que superan con éxito todas sus pruebas automatizadas.



Desglose de los Términos

- **Running (En ejecución):** La funcionalidad debe estar integrada en el código fuente principal y ejecutándose en un entorno de software real o de pruebas. No cuenta el código que está solo en la máquina de un desarrollador o pendiente de integración.
- **Tested (Probado):** Ha superado todas las pruebas de unidad, integración y aceptación automatizadas. Si una funcionalidad falla en una sola prueba o carece de validación, **no se contabiliza**.
- **Features (Funcionalidades):** Se miden características o historias de usuario completas que aportan valor de negocio real al cliente, no tareas técnicas, líneas de código o componentes parciales.

DIFERENCIA ENTRE VELOCIDAD Y RTF:

- **Velocidad (Story Points):** Mide el ritmo interno de trabajo del equipo basándose en sus propias estimaciones relativas. Es útil para planificar la capacidad del Sprint, pero no demuestra por sí sola la calidad o el funcionamiento real del producto.
- **RTF:** Mide la entrega de valor real y verificable. No le importan las estimaciones ni los puntos de historia, sino cuántas funcionalidades están **verificadas y funcionando** en el software en un momento determinado.